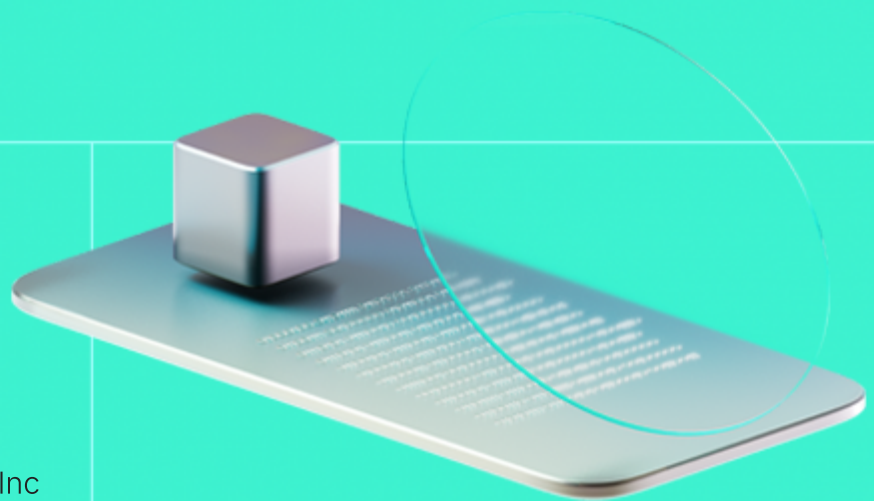




Smart Contract Code Review and Security Analysis Report for Datamine Network Inc.



Customer: Datamine Network Inc

Date: 13/07/2026

We express our gratitude to the Datamine Network Inc. team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Datamine is a multi-token DeFi "FLUX economic model" protocol targeting Arbitrum L2, with Ethereum L1 counterparts bridged via the Arbitrum token bridge, in which a base ERC-777 token is locked into a token contract to continuously mint a secondary ERC-777 token over time, with burning the secondary token driving a reward multiplier. The key user-facing actions are locking base tokens, minting the secondary token, and burning it for a higher mint multiplier.

Document

Name	Smart Contract Code Review and Security Analysis Report for Datamine Network Inc.
Audited By	Olesia Bilenka; Ivan Bondar
Approved By	Khrystyna Tkachuk
Website	https://datamine.network/
Changelog	06/07/2026 - Preliminary Report 13/07/2026 - Final Report
Platform	Arbitrum
Language	Solidity
Tags	Fungible Token; Yield Farming; Vault; Bridge; Factory;
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Datamine-Crypto/white-paper
Commit	28c902f
Deployed	FLUX(L2) Deployed address: 0xF80D589b3Dbe130c270a69F1a69D05
Contracts	0f268786Df ArbiFLUX Deployed address: 0x64081252c497FCfeC247a664e9D10Ca8eD71b276 Lockquidity Deployed address: 0x454F676D44DF315EEf9B5425178d5a8B524CEa03

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

19	0	19	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	3
Low	5

Vulnerability	Severity	Status
F-2026-17945 - Unprotected sweep Enables Atomic Sandwich Extraction of Protocol LOCK	High	Accepted
F-2026-17966 - Unbounded burnToAddress Credit Inflates globalRatio, Enabling Sustained Reward Denial	High	Accepted
F-2026-17949 - Reception-Ack Requirement Strands Bridge Deposits and Locked Principal for Contract Recipients	Medium	Accepted
F-2026-17950 - Emission Schedule Miscalibrated for Arbitrum Block Cadence	Medium	Accepted
F-2026-17967 - Reward Multipliers Read at Mint Time Are Applied Retroactively to the Whole Window	Medium	Accepted
F-2026-17944 - Silent No-Op in preventRecursion Modifier Results in Undetected Failed Transactions	Low	Accepted
F-2026-17951 - Delegated Minter in mintToAddress Redirects Accrued Rewards	Low	Accepted
F-2026-17955 - Misleading burnToAddress Function Transfers Tokens to Vault Instead of Burning	Low	Accepted
F-2026-17962 - LockquidityVault init Function Lacks Access Control	Low	Accepted
F-2026-17971 - Integer Division Truncation in getMintAmount Results in Zero Rewards for Small Lock Amounts	Low	Accepted
F-2026-17943 - Inconsistent, Floating, and Outdated Pragma Versions Reduce Code Quality and Portability	Info	Accepted
F-2026-17947 - Misleading NatSpec Comment on _percentBurnMultiplier Constant	Info	Accepted

Vulnerability	Severity	Status
F-2026-17957 - Incorrect Token Reference in Code Comment Reduces Code Clarity	Info	Accepted
F-2026-17959 - Vestigial ALREADY_INIT Guard Is Always True in Constructor	Info	Accepted
F-2026-17961 - Missing SafeMath in LockquidityVault Arithmetic Operations	Info	Accepted
F-2026-17963 - Misleading Variable Name damBalance in View Functions	Info	Accepted
F-2026-17968 - State Variables in Flux Contract Should Be Declared Immutable	Info	Accepted
F-2026-17970 - Redundant Array Allocation in _swap Function	Info	Accepted
F-2026-17972 - Incomplete Same-Block Guard in unlock Allows Immediate Relock	Info	Accepted

Documentation quality

- Functional requirements are detailed.
 - Project overview is detailed
 - All roles in the system are described.
 - Use cases are described and detailed.
 - For each contract, all futures are described.
 - All interactions are described.
- Technical description is detailed.
 - Run instructions are provided.
 - Technical specification is provided.
 - The NatSpec documentation is sufficient.

Code quality

- The development environment is configured.
- The project uses an older Solidity version (0.6.x).

Test coverage

The project is covered by a Hardhat 3 test suite of **119 test cases**. Branch coverage is not reported, as Hardhat 3's coverage engine does not emit branch data.

- Deployment and basic user interactions are covered with tests; negative-path cases are largely missed.
- The **Flux** L2 bridge token has no tests.
- Tests live in a separate [repository](#) from the audited code, which is a verified-equivalent copy of the tested contracts.

Table of Contents

System Overview	7
Privileged Roles	7
Potential Risks	9
Findings	11
Vulnerability Details	11
Disclaimers	67
Appendix 1. Definitions	68
Severities	68
Potential Risks	68
Appendix 2. Scope	69
Appendix 3. Additional Valuables	70

System Overview

The protocol implements a two-tier staking architecture where each layer follows a lock-mint pattern. At the base layer, FLUX tokens (bridged from Ethereum L1 via the Arbitrum gateway) are locked in the **ArbiFluxToken** contract to passively mint ArbiFLUX tokens. At the second layer, ArbiFLUX tokens are locked in the **LockquidityToken** contract to mint LOCK tokens. Both staking contracts share the same core mechanics: users lock tokens via `lock`, mint rewards via `mintToAddress`, and unlock via `unlock`. A delegated minter address can be specified at lock time to allow third-party minting on behalf of the lock holder.

Minting rewards are calculated based on the locked token amount, the number of blocks elapsed since the last mint, and two multiplicative bonuses. The time multiplier scales from 1x to 3x over a 28-day period following a 24-hour initial waiting period. The burn multiplier scales from 1x to 10x based on the ratio of burned tokens attributed to an address relative to the global network ratio. In **ArbiFluxToken**, burned tokens are destroyed via `burnToAddress` while incrementing the recipient's burn credit. In **LockquidityToken**, the `burnToAddress` function transfers LOCK tokens to the **LockquidityVault** rather than destroying them, allowing the vault to provide liquidity to Uniswap V2 via an optimal one-sided zap mechanism.

All three token contracts implement the ERC-777 standard with ERC-20 backward compatibility, registering interfaces via the ERC-1820 registry. The **Flux** L2 token implements the IArbToken interface for bridge mint/burn operations restricted to the Arbitrum gateway. Re-entrancy protection is implemented via a per-address mutex pattern in **ArbiFluxToken** and **LockquidityToken**, and same-block restrictions prevent lock/unlock/mint actions from occurring in a single block. The **LockquidityFactory** handles deployment of both the **LockquidityToken** and **LockquidityVault** contracts with atomic initialization.

- **arbiFlux.sol** — Implements the **ArbiFluxToken** contract, an ERC-777 token that accepts FLUX token lock-ins and mints ArbiFLUX rewards based on block-elapsed time multipliers and burn multipliers. Provides `lock`, `unlock`, `mintToAddress`, and `burnToAddress` entry points.
- **fluxL2.sol** — Implements the **Flux** contract, an ERC-777 token representing FLUX on Arbitrum L2. Exposes `bridgeMint` and `bridgeBurn` functions restricted to the L2 gateway for cross-chain token transfers via the Arbitrum bridge.
- **lockquidity.sol** — Contains **LockquidityToken** (an ERC-777 token that locks ArbiFLUX to mint LOCK with the same multiplier mechanics as **ArbiFluxToken**), **LockquidityVault** (receives LOCK tokens via `burnToAddress` and provides Uniswap V2 liquidity via optimal swaps), and **LockquidityFactory** (deploys and atomically initializes both contracts).

Privileged roles

ArbiFluxToken (arbiFlux.sol)

- **minterAddress** (per-address delegation): Designated address authorized to mint ArbiFLUX tokens on behalf of a FLUX lock-in holder.
 - Can call `mintToAddress` to mint ArbiFLUX tokens from the delegator's locked-in position to any specified target address.

Flux (fluxL2.sol)



- **I2Gateway**: L2 bridge gateway address set at construction; controls token supply for cross-chain bridging.
 - Can call `bridgeMint` to mint FLUX tokens to any account when tokens are bridged from L1.
 - Can call `bridgeBurn` to burn FLUX tokens from any account when tokens are withdrawn to L1.

LockquidityToken (lockquidity.sol)

- **minterAddress** (per-address delegation): Designated address authorized to mint LOCK tokens on behalf of an ArbiFLUX lock-in holder.
 - Can call `mintToAddress` to mint LOCK tokens from the delegator's locked-in position to any specified target address.

LockquidityVault (lockquidity.sol)

- **First initializer**: The first caller to invoke `init` before `isInitialized` is set; determines the token address used by the vault.
 - Can call `init` once to set the LOCK token address and permanently enable the vault.

LockquidityFactory (lockquidity.sol)

- No privileged roles. Contract is deployed via constructor only, creating the **LockquidityVault** and **LockquidityToken** instances with atomic initialization. All configuration is fixed at deployment time.

Potential Risks

Centralization and Trust Assumptions

- **L2 Gateway Privileged Control:** The **Flux** L2 token grants exclusive minting and burning authority to the `L2Gateway` address via the `onlyGateway` modifier on `bridgeMint` and `bridgeBurn`. The gateway address is set at construction with a zero-address initialization check preventing reassignment; the security of the L2 token supply is entirely dependent on the trustworthiness and security of the external Arbitrum bridge infrastructure.
- **Permanently Locked Liquidity Provider Tokens:** The **LockquidityVault** contract's `_addLiquidity` function sends LP tokens to `address(this)` with no mechanism to withdraw or transfer them. Any LP tokens received from Uniswap V2 liquidity provisioning are permanently locked in the vault, eliminating governance or recovery options for the protocol.

External Dependencies

- **ERC1820 Registry Dependency:** All three token contracts (**ArbiFluxToken**, **Flux**, and **LockquidityToken**) depend on the global ERC1820 Registry at the hardcoded address `0x1820a4B7618BdE71Dce8cdc73aAB6C95905faD24`. The tokens register their ERC777 interfaces during construction and query the registry for send/receive hook implementers on every transfer. If the registry becomes unavailable or behaves unexpectedly, token operations may fail or behave unpredictably.
- **Hardcoded Uniswap V2 Arbitrum Addresses:** The **LockquidityVault** contract hardcodes Uniswap V2 deployment addresses for Arbitrum (`FACTORY: 0xf1D7CC64Fb4452F05c498126312eBE29f30Fbcf9` , `ROUTER: 0x4752ba5DBc23f44D87826276BF6Fd6b1C372aD24` , `WETH: 0x82aF49447D8a07e3bd95BD0d56f35241523fBab1`). If these Uniswap contracts are upgraded, deprecated, or if the protocol is deployed to a different network, all vault swap and liquidity functionality will fail.
- **Dependency on External Token Addresses:** The **LockquidityFactory** constructor hardcodes the **ArbiFLUX** token address `0x64081252c497FCfeC247a664e9D10Ca8eD71b276` for the **LockquidityToken** dependency. The **ArbiFluxToken** similarly depends on an external FLUX token address provided at construction. The behavior and security properties of these external tokens are outside the scope of this audit.

Token Standard Compliance

- **ERC777 Hook Reentrancy Surface:** All three token contracts implement ERC777, which invokes external `tokensToSend` and `tokensReceived` hooks via the ERC1820 registry during transfers. While **ArbiFluxToken** and **LockquidityToken** implement a custom `preventReursion` mutex modifier to mitigate reentrancy, the **Flux** L2 token does not implement such protection on its `bridgeMint` and `bridgeBurn` functions, relying solely on the `onlyGateway` modifier for access control.

Integration Risks

- **Uniswap Swap with Minimal Slippage Protection:** The **LockquidityVault** function `_swap` calls `swapExactTokensForTokens` with `amountOutMin` set to `1`, and `_addLiquidity` calls `addLiquidity` with minimum amounts of `0, 0`. These parameters provide negligible slippage protection, making the

`sweep` operation vulnerable to sandwich attacks and unfavorable trade execution when processing accumulated LOCK tokens.

- **Restrictive Token Reception Logic:** The `tokensReceived` implementations in **ArbiFluxToken** and **LockquidityToken** require that the `operator` parameter equals `address(this)`, meaning only the contract itself can send tokens to itself. If FLUX or ArbiFLUX tokens are sent directly to these contracts by users via external `send` or `operatorSend` calls, those transactions would be rejected by the hook and revert.

Economic and Game Theory Risks

- **Burn Multiplier Relative to Global State:** The burn multiplier calculation in **ArbiFluxToken** and **LockquidityToken** (`getAddressBurnMultiplier`) compares an individual address's burn ratio against the global burn ratio. Large participants who burn significant amounts can dilute the global ratio, reducing the effective multiplier for smaller participants. The formula computes `min(_maxBurnMultiplier, myRatio * _percentMultiplier / globalRatio + baseMultiplier)`, meaning early movers or large stakers can establish favorable positions that are difficult for later entrants to match.
- **Vault Initialization Front-Running Window:** The **LockquidityVault** `init` function uses a simple `require(!isInitialized)` check without access control. While the **LockquidityFactory** constructor atomically deploys and initializes the vault, standalone deployment of **LockquidityVault** would create a window where any address could call `init` with an arbitrary token address, potentially rendering the vault unusable or malicious.

Cross-Chain Consistency

- **L1/L2 Token Supply Synchronization:** The **Flux** L2 token relies on the Arbitrum bridge gateway to maintain supply parity with L1. The `bridgeMint` function increases L2 supply when tokens are deposited on L1, and `bridgeBurn` decreases L2 supply for withdrawals. If message passing between L1 and L2 experiences delays, failures, or reorgs, temporary supply inconsistencies may occur until finality is achieved on both chains.

Findings

Vulnerability Details

F-2026-17945 - Unprotected sweep Enables Atomic Sandwich Extraction of Protocol LOCK - High

Description:

In **LockquidityVault**, the `sweep` function is external and permissionless, guarded only by a check that the vault is initialized. It reads the vault's full LOCK balance and routes it through `zap`, which sizes and executes a swap and liquidity deposit against live pair reserves in the same call frame, with no trusted price reference and no slippage protection.

```
function zap(address _tokenA, address _tokenB, uint256 _amountA) internal {
    // ...
    (uint256 reserve0, uint256 reserve1,) = IUniswapV2Pair(pair).getReserves(
);
    // swapAmount derived from live reserves in the same frame
    swapAmount = getSwapAmount(reserve0, _amountA);
    _swap(_tokenA, _tokenB, swapAmount);
    _addLiquidity(_tokenA, _tokenB);
}

function _swap(address _from, address _to, uint256 _amount) internal {
    // ...
    IUniswapV2Router(ROUTER).swapExactTokensForTokens(_amount, 1, path, address(
ss(this)), block.timestamp);
}
```

The swap accepts `amountOutMin` of 1, and the liquidity add passes both minimums as 0, so any output ratio is accepted. Because the swap size is derived from the same reserve an attacker just manipulated, inflating the LOCK reserve makes the vault swap more LOCK at the worse price, amplifying the loss. The caller controls timing and transaction context, so the attacker originates a single atomic transaction that skews the pool, calls `sweep`, then restores the pool and repays a flash loan, capturing the price-impact differential. Arbitrum's lack of a public mempool provides no mitigation since no front-running of a victim is required. The `deadline` value of the current block timestamp always satisfies the router check and adds no protection.

Since the vault accumulates all burned LOCK with no withdrawal path, the extracted value is irrecoverable and the leak repeats on every accrual and

sweep cycle. Severity escalates toward catastrophic when the vault balance is large relative to a thin pool, where a single attack can remove more than a tenth of the swapped value.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate: 5/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Medium

Severity: High

Recommendations

Remediation:

Enforce real slippage bounds instead of the current fixed floors of 1 and 0. Two options:

- Require the caller to pass a minimum WETH output and minimum LP amount, and revert when either is unmet.
- Derive those minimums on-chain from a manipulation-resistant price such as a time-weighted average or an external oracle, and bound the allowed deviation.

Restricting `sweep` to an authorized keeper further removes the attacker's control over timing.

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is a known design constraint of the decentralized, permissionless architecture of the Lockquidity vault.

1. Rationale for Permissionless Design

To maintain the protocol's strict definition of absolute decentralization ("no admin keys, no DAO, no servers/central entities"), the `sweep()` function was designed to be permissionless. Anyone can call it at any time to execute the liquidity injection without relying on a centralized keeper bot.

2. Economic Mitigations & Friction

While the sandwich vector is theoretically possible if the vault holds a very large LOCK balance, it is mitigated by several factors in practice:

- **Uniswap Trading Fees:** An attacker must perform both an entry swap and an exit swap, incurring Uniswap's 0.3% fee twice (0.6% total pool trading friction) in addition to Arbitrum gas costs.
- **Frequent Sweeps:** In production, the vault is swept by a bot every few hours. This keeps the accumulated vault amounts very small relative to the Uniswap pool size.
- **Negligible Profit:** Due to the small swept amounts, any potential slippage profit from a sandwich attack is extremely low—on the order of a few cents. This is too small to motivate rational actors when transaction and swap fee overhead are considered.
- **Decentralized Sweeping Incentive:** Having third-party bots trigger the sweep is a deliberate design choice. If external bots are incentivized to call `sweep()`, it decentralizes the sweeping mechanics further (which is a core goal). The protocol is willing to pay a small premium (in the form of minor slippage) to achieve this fully decentralized, serverless automation.
- **Permanently Locked LP:** All Uniswap LP tokens generated during the `sweep()` are minted directly to `address(this)` (the vault contract). Since the vault contains no transfer, approval, or withdrawal logic for LP tokens, these LP tokens are permanently locked (effectively burned) inside the protocol, securing long-term liquidity depth.

3. Contract Immutability

The `LockquidityVault` contract is deployed as an immutable protocol on Arbitrum (L2) with no admin controls, owner keys, or upgrade path.

Evidences

Reproduce:

PoC Steps (single attacker-originated transaction):

- Seed LOCK/WETH pool at fair price (1,000,000 LOCK / 1,000 WETH = 0.001 WETH/LOCK); vault holds 500,000 protocol-owned LOCK.
- Attacker flash-borrows 2,000,000 LOCK and swaps LOCK→WETH, skewing the pool LOCK-heavy / WETH-poor.
- Attacker calls `vault.sweep()` — the vault reads the manipulated reserves, sizes the swap against them, and dumps LOCK for floor-priced WETH (`amountOutMin=1`), then `addLiquidity(min=0,0)`.
- Attacker swaps WETH→LOCK to restore the pool.
- `assertGt(attackerLock, borrowed)` — risk-free LOCK profit; and `assertLt(vaultValueAfter, fairValue/2)` — vault lost >50%.

PoC Code:

```

function test_sweep_sandwich_extracts_protocol_lock() public {
    uint256 fairVaultValue = (VAULT_LOCK * INIT_WETH_RESERVE) / INIT_LOCK_RESERVE;

    // Attacker takes a flash loan of LOCK (simulated by mint).
    lock.mint(attacker, BORROW);

    vm.prank(attacker);
    lock.approve(ROUTER, uint256(-1));
    vm.prank(attacker);
    MockERC20(WETH).approve(ROUTER, uint256(-1));

    // 1. Skew the pool LOCK-heavy / WETH-poor.
    address[] memory sell = new address[](2);
    sell[0] = address(lock);
    sell[1] = WETH;
    vm.prank(attacker);
    MockUniV2Router(ROUTER).swapExactTokensForTokens(BORROW, 1, sell, attacker, block.timestamp);

    // 2. Permissionless sweep against the manipulated reserves.
    vm.prank(attacker);
    vault.sweep();

    // 3. Unwind: swap WETH back to LOCK.
    uint256 wethBack = MockERC20(WETH).balanceOf(attacker);
    address[] memory buy = new address[](2);
    buy[0] = WETH;
    buy[1] = address(lock);
    vm.prank(attacker);
    MockUniV2Router(ROUTER).swapExactTokensForTokens(wethBack, 1, buy, attacker, block.timestamp);

    uint256 attackerLock = lock.balanceOf(attacker);
    uint256 vaultValueAfter = _vaultValueInWeth();

    uint256 attackerProfitWeth = ((attackerLock - BORROW) * INIT_WETH_RESERVE) / INIT_LOCK_RESERVE;
    uint256 vaultLossWeth = fairVaultValue - vaultValueAfter;

    emit log_named_uint("borrowed LOCK", BORROW);
    emit log_named_uint("attacker LOCK after unwind", attackerLock);
    emit log_named_uint("attacker net LOCK profit", attackerLock - BORROW);
    emit log_named_uint("attacker profit (WETH-equiv)", attackerProfitWeth);
    emit log_named_uint("vault value (WETH) fair", fairVaultValue);
    emit log_named_uint("vault value (WETH) after attack", vaultValueAfter);
    emit log_named_uint("vault loss (WETH-equiv)", vaultLossWeth);
}

```

```
[PASS] test_sweep_sandwich_extracts_protocol_lock() (gas: 421623)
```

```

borrowed LOCK: 2000000000000000000000000000000000
attacker LOCK after unwind: 2328654981328760402377236
attacker net LOCK profit: 328654981328760402377236
vault value (WETH) fair: 5000000000000000000000000000000000
vault value (WETH) after attack: 160846125332105400643

```

PoC_LockquidityVault_sweep_sandwich.t.sol

F-2026-17966 - Unbounded burnToAddress Credit Inflates globalRatio, Enabling Sustained Reward Denial - High

Description:

In **ArbiFluxToken** and **LockquidityToken**, the burn multiplier is `myRatio / globalRatio` plus a 1x floor (capped at 10x), where `getGlobalRatio` returns `globalBurnedAmount * 1e10 / globalLockedAmount`. The `burnToAddress` function credits an unbounded `burnedAmount` to any locked target, with no relation enforced between that credit and the target's locked `amount`.

```
function burnToAddress(address targetAddress, uint256 amount)
    preventRecursion
    requireLocked(targetAddress, true)
public {
    // ...
    targetAddressLock.burnedAmount = targetAddressLock.burnedAmount.add(amount);
    globalBurnedAmount = globalBurnedAmount.add(amount);
    // ...
}
```

Because a position's `burnedAmount` is decoupled from its locked `amount`, a negligibly-locked position can carry arbitrary burn weight. Crediting a large burn against a minimally-locked target inflates `globalBurnedAmount` while leaving `globalLockedAmount` effectively unchanged, driving `globalRatio` upward. As the multiplier is relative, this suppresses every honest participant's multiplier without altering their own state. The vector is aggravated by two properties: `burnedAmount` is never cleared on `unlock` and is re-added to `globalBurnedAmount` on each `lock`, making a one-time burn replayable at negligible cost, and `burnToAddress` carries no `preventSameBlock` guard.

On the LOCK layer the impact escalates from dilution to denial: once `globalRatio` exceeds `myRatio * 1e4`, the ratio term truncates to zero and the multiplier collapses to the additive floor of `0.0001x`, so a sufficiently large burn suppresses every honest locker's reward multiplier to near zero and the banked-credit replay sustains that state indefinitely. On the ArbiFLUX layer, where the floor is `1x`, the same vector yields milder dilution. The effect is strongest at low total value locked.

Assets:

- `arbiFlux.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- `lockquidity.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Cap a position's `burnedAmount` contribution to `globalBurnedAmount` relative to its locked `amount`, so a negligibly-locked position cannot carry disproportionate burn weight and thus cannot move `globalRatio`. Reset or forfeit `burnedAmount` on `unlock` so banked credit cannot be re-injected against a fresh lock.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** to protect the integrity of the validator rewards ecosystem and reward long-term participation.

1. The Rationale Behind Not Clearing `burnedAmount`

In the Datamine ecosystem, locking is dynamic, but burned amount is permanent progress.

- **Global Accounting Cleanup:** We kept this functionality on purpose to make sure that when users EXIT (unlock) the system fully their burns are not counted towards the global result. Subtracting their `burnedAmount` contribution from `globalBurnedAmount` is crucial: it ensures that inactive/exited validators do not continue to inflate `globalRatio` and dilute the active validators.
- **Saving Progress:** However, the validator's personal `burnedAmount` progress is saved locally. If the validator chooses to re-enter (re-lock), their historic burn progress is restored. Forfeiting this progress would heavily penalize long-term validators who occasionally need to re-lock or adjust their stakes.

2. Economic Feasibility & Scale of "Free Replay"

The audit finding highlights that an attacker can "replay" dilution by locking/unlocking 1 wei to inject/withdraw their banked burn credit.

- **No Free Credit:** An attacker cannot inject a "banked credit" unless they **actually burned** the tokens in the first place. Since burning permanently destroys tokens (representing capital), this credit is backed by proof-of-burn.
- **Acceptable "Early Start" Scenario:** At this early scale of the ecosystem, you cannot meaningfully change the balance. The current numbers demonstrate that manipulation is not economically meaningful:
 - **LOCK Current Supply:** 30,131.51096831 LOCK / \$71,326.96 USD
 - **LOCK Burned:** 36,928.74187929 LOCK / \$87,417.28 USD (122.56% of minted LOCK)
 - **LOCK Burn Ratio:** 0.14018 LOCK / 1 ArbiFLUX
 - **LOCK Available Uniswap Liquidity:** 28,636.57 LOCK / \$67,614.56 USD (95.04% of current supply)
 - **LOCK / ETH Uniswap Available ETH:** 38.92 ETH / \$67,614.56 USD
 - **LOCK / ETH Total Liquidity:** \$135,229.12 USD
 - **LOCK Realtime Market Cap:** \$71,144.30 USD
 - **ArbiFLUX Available Uniswap Liquidity:** 60,305.43 ArbiFLUX / \$2,641.89 USD (15.04% of L2 supply)
 - **ArbiFLUX Uniswap Available ETH:** 1.52 ETH / \$2,641.89 USD
 - **ArbiFLUX / ETH Total Liquidity:** \$5,283.78 USD
 - **ArbiFLUX Realtime Market Cap (on L2):** \$6,024.43 USD (Circulating) / \$17,565.04 USD (Total)

At this scale, you cannot meaningfully change the balance. An attacker who has already burned a massive amount of LOCK or FLUX has already contributed significant value to the ecosystem and is entitled to the corresponding ratio weight when active. Toggling their active validator status (by locking/unlocking 1 wei) does not give them an unfair advantage; it simply toggles their own historical burn contribution, which they fully paid for.

Furthermore, as soon as the first real burn/lock occurs, the burn and lock amounts scale into the high (10^{18}) (wei) ranges where this ceases to be a mathematical issue within 5 minutes of launching the system. The protocol has already successfully run in production for over 1 year and 262 days without any such vulnerability issues.

Evidences

Unbounded burnToAddress Credit Inflates globalRatio

Reproduce:

PoC Steps:

- Establish an honest baseline: Bob locks `10e18` FLUX and burns `5e18` toward himself (`myRatio = 5e9`); Carol supplies the rest of the network SO `globalLocked = 1000e18`, `globalBurned = 100e18`, `globalRatio = 1e9`.

- Confirm Bob's burn multiplier is **6.0x** (`getAddressBurnMultiplier == 60000`).
- Attacker Eve locks **1 wei** of FLUX, then in the same block burns **900e18** toward it via `burnToAddress` (no `preventSameBlock`).
- Observe `getGlobalRatio()` jumps to `9999999999` (~1e10) while Bob's own `getAddressRatio` is still `5e9` — his multiplier collapses to **1.5x** (`15000`).
- Eve calls `unlock()`: `globalBurned` is refunded but her `burnedAmount` (900e18) is **not** reset, so Bob recovers to 6.0x.
- Eve re-locks **1 wei** with **no new burn** → the banked 900e18 is re-injected, `globalBurned` returns to `1000e18`, and Bob is re-suppressed to 1.5x — proving the grief is replayable for free.

PoC Code:

```
function test_burnToAddress_inflates_globalRatio_and_dilutes_rewards() public
{
    // globalLocked = 1000e18, globalBurned = 100e18, globalRatio = 1e9
    _lock(bob, 10e18);
    _lock(carol, 990e18);
    _burnSelf(bob, 5e18); // myRatio = 5e18*1e10/10e18 = 5e9
    _burnSelf(carol, 95e18);

    assertEquals(arbi.getGlobalRatio(), 1e9, "global ratio baseline");
    assertEquals(arbi.getAddressRatio(bob), 5e9, "bob ratio baseline");

    uint256 bobBefore = arbi.getAddressBurnMultiplier(bob);
    assertEquals(bobBefore, 60000, "bob earns 6.0x"); // 60000 = 6.0x

    // 1-wei lock + 900e18 burn in one block (burnToAddress has no preventSameBlock)
    vm.roll(101);
    _lock(eve, 1);
    _burnSelf(eve, 900e18);

    assertEquals(arbi.getGlobalRatio(), 9999999999, "ratio driven to ~1e10");
    assertEquals(arbi.getAddressRatio(bob), 5e9, "bob's own ratio unchanged");

    uint256 bobDuring = arbi.getAddressBurnMultiplier(bob);
    assertEquals(bobDuring, 15000, "bob suppressed to 1.5x"); // 15000 = 1.5x
    assertTrue(bobDuring < bobBefore, "honest multiplier suppressed");

    vm.roll(102);
    vm.prank(eve);
    arbi.unlock(); // refunds globalBurned, keeps banked burnedAmount
    assertEquals(arbi.getAddressBurnMultiplier(bob), 60000, "recovers after unlock");

    vm.roll(103);
    _lock(eve, 1); // re-injects the same 900e18, no new burn
}
```

```
assertEq(arbi.globalBurnedAmount(), 1000e18, "banked burn re-injected");
assertEq(arbi.getAddressBurnMultiplier(bob), 15000, "re-suppressed for fr
ee");
}
```

Results:

Result: Bob's reward multiplier is driven 6.0x → 1.5x entirely by another address's state, repeatable indefinitely.

```
Ran 1 test for test/PoC_ArbiFlux_burnToAddress_ratio_inflation.t.sol:PoC_Arbi
Flux_burnToAddress_ratio_inflation
[PASS] test_burnToAddress_inflates_globalRatio_and_dilutes_rewards() (gas: 97
1150)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.89ms (899.75µs
CPU time)
```

Files:

PoC_ArbiFlux_burnToAddress_ratio_inflation.t.sol

F-2026-17949 - Reception-Ack Requirement Strands Bridge Deposits and Locked Principal for Contract Recipients - Medium

Description:

The **Flux**, **ArbiFluxToken**, and **LockquidityToken** contracts return or mint tokens through ERC-777 paths that call the recipient reception hook with the ack flag hardcoded to true. When the recipient is a contract with no registered ERC-1820 implementer, `_mint` and `send` revert on the check that forbids a contract recipient. Most contracts (multisigs, vaults, routers, exchange deposit addresses) never register one.

On the bridge path, `bridgeMint` forwards to `_mint` while the gateway redeems the deposit's retryable ticket, so the revert blocks L2 finalization; the ticket expires in about a week and the escrowed L1 tokens stay locked with no L2 counterpart.

```
function bridgeMint(address account, uint256 amount) external virtual override
onlyGateway {
    _mint(account, amount, '', '');
}
```

On the token path, a contract can `lock` (the `operatorSend` recipient is the self-registered token contract), but `unlock` returns principal via `send` with `requireReceptionAck = true`, whose recipient is now the contract locker:

```
function unlock() /* ... */ public {
    // ...
    IERC777(_token).send(_msgSender(), amount, "");
}
```

`_callTokensReceived` finds no implementer and reverts, so a hookless contract locker reverts on every `unlock`. If it cannot register an implementer afterward, its principal is permanently unrecoverable; the same applies to `mintToAddress` minting to a hookless target. Externally owned accounts are unaffected.

Assets:

- fluxL2.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:

3/5

Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: For the bridged L2 token, prefer a plain ERC-20 so the mint path cannot revert on contract recipients, per Arbitrum's `L2GatewayToken` guidance. For `unlock` and `mintToAddress`, return funds with a non-ack transfer or restrict participation to ERC-777-aware recipients. At minimum, document the recipient-hook requirement for contract integrators.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted risk** stemming from the protocol's reliance on the ERC-777 token standard.

1. Context and Arbitrum Bridge Frontend Restrictions

This protocol was designed and deployed before Smart Accounts became prevalent. For standard operations:

- The Arbitrum bridge frontend is built to expect and handle Externally Owned Accounts (EOAs). If EOA verification is active, the frontend itself will throw an error and prevent bridging from a smart contract address that does not support the bridge.
- If a custom smart contract wants to utilize the Arbitrum bridge to transfer tokens, it **must** register an implementer for `ERC777TokensRecipient` via the ERC-1820 registry to ensure compatibility.

2. Compatibility Layer

To support Smart Accounts (like MetaMask's new default smart accounts) that do not implement the ERC-1820 registry by default, the protocol utilizes wrapper or helper contracts to bridge the gap. For any custom contract integration, registering the appropriate ERC-1820 interface remains a prerequisite to ensure compatibility with our ERC-777 token rules.

F-2026-17950 - Emission Schedule Miscalibrated for Arbitrum Block Cadence - Medium

Description:

In **ArbiFluxToken**, every block-based constant is calibrated to a "1 block = 15 seconds" assumption inherited from the pre-Merge Ethereum design, including the reward start delay, the maximum time-bonus window, the base per-block emission rate, and the failsafe window. These values feed `getAddressTimeMultiplier` and the emission math in `getMintAmount`. The same constants and calibration are reused unchanged by the LOCK layer in **LockquidityToken**, so the deviation applies to the shared reward engine, not to a single token.

```
function getAddressTimeMultiplier(address targetAddress) public view returns(
uint256) {
    // ...
    uint256 targetBlockNumber = targetAddressLock.blockNumber.add(_startTimeReward);
    // ...
    uint256 blockDiff = block.number.sub(targetBlockNumber).mul(_targetBlockMultiplier).div(_maxTimeReward).add(_percentMultiplier);
    uint256 timeMultiplier = Math.min(_maxTimeMultiplier, blockDiff);
    return timeMultiplier;
}
```

On Arbitrum, `block.number` tracks the L1 block number estimate, which advances at the roughly 12-second Ethereum cadence, not an L2 transaction counter. Post-Merge L1 blocks are about 12 seconds rather than the assumed 15, so the block height advances about 1.25x faster per unit real time. The emission rate therefore over-delivers by roughly 25%, while the bonus ramp reaches its thresholds sooner in real time: the time bonus starts after about 19.2 hours instead of 24, and the maximum 3x bonus is reached after about 22.4 days instead of 28. All governing constants are immutable or constant with no setter, so the deviation from the documented 24-hour and 28-day schedule is permanent.

There is no principal loss and no cross-user theft; the deviation is confined to the protocol's own emission rate and bonus timing, and it affects all participants uniformly.

Assets:

- `arbiFlux.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- `lockquidity.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Recompute the reward start delay, the maximum time window, the failsafe duration, and the per-block emission against the actual Arbitrum block cadence in both **ArbiFluxToken** and **LockquidityToken**.

Alternatively, migrate the schedule to `block.timestamp`, which tracks wall-clock time far better than `block.number` over multi-day windows on L2.

If the constants are already deployed and immutable, document the true real-time schedule for users.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted risk** stemming from the protocol's reliance on the ERC-777 token standard.

1. Context and Arbitrum Bridge Frontend Restrictions

This protocol was designed and deployed before Smart Accounts became prevalent. For standard operations:

- The Arbitrum bridge frontend is built to expect and handle Externally Owned Accounts (EOAs). If EOA verification is active, the frontend itself will throw an error and prevent bridging from a smart contract address that does not support the bridge.
- If a custom smart contract wants to utilize the Arbitrum bridge to transfer tokens, it **must** register an implementer for `ERC777TokensRecipient` via the ERC-1820 registry to ensure compatibility.

2. Compatibility Layer

To support Smart Accounts (like MetaMask's new default smart accounts) that do not implement the ERC-1820 registry by default, the protocol utilizes wrapper or helper contracts to bridge the gap. For any custom contract integration, registering the appropriate ERC-1820 interface

remains a prerequisite to ensure compatibility with our ERC-777 token rules.

F-2026-17967 - Reward Multipliers Read at Mint Time Are Applied Retroactively to the Whole Window - Medium

Description:

In **ArbiFluxToken** and **LockquidityToken**, `getMintAmount` computes the reward as `amount * blocksMinted * burnMult * timeMult / divisors`, applying both multipliers as scalars read from current state instead of integrating them over the accrual window.

```
function getMintAmount(address targetAddress, uint256 targetBlock) public view
returns(uint256) {
    // ...
    uint256 blocksMinted = targetBlock.sub(targetAddressLock.lastMintBlockNum
ber);
    uint256 blocksMintedByAmount = amount.mul(blocksMinted);
    uint256 burnMultiplier = getAddressBurnMultiplier(targetAddress);
    uint256 timeMultiplier = getAddressTimeMultiplier(targetAddress);
    uint256 fluxAfterMultiplier = blocksMintedByAmount.mul(burnMultiplier).di
v(_percentMultiplier).mul(timeMultiplier).div(_percentMultiplier);
    // ...
}
```

`getAddressTimeMultiplier` derives its value from `block.number`, not `targetBlock`, and `getAddressBurnMultiplier` reads the current `burnedAmount` and `globalRatio`; both are then multiplied against every block in the window, including early blocks that accrued at a lower multiplier. Since `burnToAddress` writes neither `blockNumber` nor `lastMintBlockNumber`, it does not trip the `preventSameBlock(sourceAddress)` guard on `mintToAddress`, so a burn and a mint in one transaction apply fresh burn weight to the whole window. The intended "lock longer earns up to 3x" bonus is a time-weighted integral, whereas the implementation computes `multiplier(now) * sum(blocks)`.

A locker who waits until the time multiplier caps at 3x and then mints one window applies 3x across the full duration, roughly 1.5x more than an incremental minter earns over the ramp. With no global emission cap, back-loading minters systematically over-extract relative to the documented schedule. No principal is at risk.

Assets:

- `arbiFlux.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- `lockquidity.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Ensure each accrued block is rewarded at the multiplier that applied while it was earned, rather than at the current-state multiplier, so that delaying a mint or back-loading a burn cannot retroactively uplift earlier blocks. If the current behavior is intended, document the mint-time evaluation and its distributional effect explicitly.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** designed to optimize gas efficiency and introduce favorable economic incentives for validators.

1. Gas Optimization and O(1) Scalability

Calculating time-weighted or checkpointed multipliers block-by-block would require loop iterations or complex contract checkpointing storage writes.

- On Ethereum Mainnet (L1) and rollup chains (L2), loop-based historical calculations consume substantial gas, which could easily exceed the value of the validator's accrued mint rewards.
- Computing the multiplier as a simple, constant-time **O(1)** evaluation at the moment of minting avoids loop execution, keeping transaction fees as low as possible for all users.

2. Retroactive Burn Incentives ("Supercharging")

Allowing retroactive multipliers is a core game-theoretic mechanic of the ecosystem. It enables validators to accumulate rewards over time and then execute a single large burn to "supercharge" their entire pending block window. This immediate payoff provides a strong psychological and economic incentive for validators to commit capital to burning.

3. Verification in Unit Tests

This behavior is explicitly covered and expected in our test suites:

- Should allow multiplier retroactivity / supercharging in the same block in both `FluxTokenAttackTests.ts` and `LockTokenAttackTests.ts` verifies and guards this dynamic.

4. Contract Immutability

The deployed contracts are completely decentralized and immutable. This retroactive multiplier mechanism has operated securely and transparently in production for over 5 years (on L1 FLUX) and over 1 year and 262 days (on L2 LOCK).

Evidences

Reward Multipliers Read at Mint Time Applied Retroactively

Reproduce:

PoC Steps:

Part A — time multiplier back-dated over the whole window:

- Alice and Dave take identical `1000e18` positions at the same block (no burns → burn multiplier constant 1x).
- Alice mints once per block for 100 blocks — each 1-block chunk is credited at that block's multiplier, tracking the intended 1x→3x ramp.
- Dave mints nothing until the ramp caps (`getAddressTimeMultiplier == 30000`, 3x), then mints the entire `[START, START+100]` window in one call.
- Compare balances: Dave $\approx 3.00e15$ vs. Alice $\approx 2.01e15 \rightarrow \sim 1.49x$ over-emission for an identical position, purely by back-loading the mint.

Part B — burn multiplier back-dated via burn-then-mint in one block:

- Mallory locks `1e18`; Carol locks a passive `4e18` (dilutes `globalRatio` so Mallory's burn is worth exactly 6x).
- Age the position a full 100-block window with no minting; snapshot the mintable amount at burn multiplier 1x → `fair = getMintAmount(...)`.
- In the **same block**, Mallory burns `1e18` via `burnToAddress` (no `preventSameBlock`) → `getAddressBurnMultiplier == 60000` (6x).
- Re-read `getMintAmount` on the *identical* window → `gamed == fair * 6`: the last-block burn is back-dated 6x onto all 100 blocks that predate it.
- Call `mintToAddress` in that same block (only the source's own lock/mint block is guarded, not a fresh burn) → the 6x amount is paid out.

PoC Code:

```
function test_time_multiplier_backdated_over_whole_window() public {
    _lock(alice, L);
    _lock(dave, L);
```

```

// Alice mints each block, tracking the ramp.
for (uint256 e = 1; e <= RAMP; e++) {
    vm.roll(START + e);
    vm.prank(alice);
    arbi.mintToAddress(alice, alice, START + e);
}

// Dave takes the whole window in one mint at the 3x cap.
assertEq(arbi.getAddressTimeMultiplier(dave), 30000, "dave at 3x cap"); /
/ 30000 = 3.0x
vm.prank(dave);
arbi.mintToAddress(dave, dave, START + RAMP);

uint256 aliceMinted = arbi.balanceOf(alice);
uint256 daveMinted = arbi.balanceOf(dave);

assertTrue(daveMinted > aliceMinted, "back-loading did not over-earn");
uint256 ratioPct = daveMinted * 100 / aliceMinted;
assertTrue(ratioPct >= 148 && ratioPct <= 150, "~1.49x over-emission");
}

function test_burn_multiplier_backdated_via_burn_then_mint_one_block() public
{
    _lock(mallory, 1e18);
    _lock(carol, 4e18); // passive: makes mallory's burn worth exactly 6x

    vm.roll(START + RAMP); // age the position, no minting

    assertEq(arbi.getAddressBurnMultiplier(mallory), 10000, "1x before burn")
; // 10000 = 1.0x
    uint256 fair = arbi.getMintAmount(mallory, START + RAMP);

    // Same block: burn spikes the multiplier (burnToAddress has no preventSa
meBlock)
    _burnSelf(mallory, 1e18);
    assertEq(arbi.getAddressBurnMultiplier(mallory), 60000, "6x after last-bl
ock burn"); // 60000 = 6.0x

    // Same already-accrued window is now worth 6x.
    uint256 gamed = arbi.getMintAmount(mallory, START + RAMP);
    assertEq(gamed, fair * 6, "burn back-dated 6x onto the whole window");

    vm.prank(mallory);
    arbi.mintToAddress(mallory, mallory, START + RAMP);
    assertEq(arbi.balanceOf(mallory), gamed, "6x paid out from one-block burn
-then-mint");
}

```

Results:

Result: a single late mint applies the capped 3x across the full window (~1.49x excess), and a single last-block burn multiplies the entire aged window by 6x — both stacking multiplicatively in one mint.

```
Ran 2 tests for test/PoC_ArbiFlux_retroactive_multipliers.t.sol:PoC_ArbiFlux_
retroactive_multipliers
[PASS] test_burn_multiplier_backdated_via_burn_then_mint_one_block() (gas: 61
5829)
[PASS] test_time_multiplier_backdated_over_whole_window() (gas: 5016153)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 10.61ms (5.21ms
CPU time)
```

Files:

PoC_ArbiFlux_retroactive_multipliers.t.sol

[F-2026-17944](#) - Silent No-Op in preventRecursion Modifier Results in Undetected Failed Transactions - Low

Description:

The `preventRecursion` modifier implements reentrancy protection using a per-address mutex pattern. When a reentrant call is detected, the modifier silently skips execution of the function body instead of reverting.

```
modifier preventRecursion() {  
    if(mutex[_msgSender()] == false) {  
        mutex[_msgSender()] = true;  
        _;  
        mutex[_msgSender()] = false;  
    }  
    // Silent return if mutex is true  
}
```

In **ArbiFluxToken** and **LockquidityToken**, this modifier protects `lock`, `unlock`, `burnToAddress`, and `mintToAddress`. If reentrancy occurs via ERC777 hooks, the nested call completes successfully (doing nothing) rather than reverting. External integrations expecting standard revert-on-reentrancy behavior may incorrectly assume the operation succeeded. Gas is consumed for the outer transaction while no state change occurs, and no error is raised to signal the failure.

Assets:

- `arbiFlux.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- `lockquidity.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Replace the conditional execution with an explicit revert on reentrancy detection:

```
modifier preventReursion() {  
    require(!mutex[_msgSender()], "Reentrant call");  
    mutex[_msgSender()] = true;  
    _;  
    mutex[_msgSender()] = false;  
}
```

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** designed to safely handle ERC-777 hook interactions without unnecessarily aborting complex multi-step transactions.

1. Rationale for Silent Return (Skip vs. Revert)

When dealing with ERC-777 hooks, contract re-entry can easily happen. If a reentrant call attempts to access a recursion-guarded function, throwing an explicit revert would abort the entire outer transaction:

- **Why We Avoid Reverting:** Aborting the entire transaction undoes valid, non-harmful state changes (such as transferring tokens or spending token balances).
- **The Skip Pattern:** By silently skipping execution of the recursion-guarded logic, the contract prevents the reentrant attack vector from altering state, while still allowing the surrounding non-reentrant transaction execution to complete successfully.

2. Verified via Hook and Same-Block Re-Entry Unit Tests

This reentrancy protection, its interaction with ERC-777 hooks, and same-block re-entry flows are a core part of our testing suite:

- The complex unit tests in `DamBlockingHolderContractTests.ts` specifically showcase same-block re-entry and hook-based attack vectors.
- For example, the test `Re-Entry Test: DamBlockingHolder should prevent unlock() inside lock() with mutex AND allow send() before lock() finishes` demonstrates that while re-entry is bypassed silently (allowing a secondary `send()` within the same transaction), the contract remains completely secure against unwanted side-effects (such as nested `unlock()` calls during a `lock()` execution).

3. Contract Immutability

The contracts are deployed as decentralized, immutable protocols on-chain, and this reentrancy mutex logic has successfully and safely

protected the contracts for over 5 years on Ethereum L1 and over 1 year and 262 days on Arbitrum L2.

[F-2026-17951](#) - Delegated Minter in mintToAddress Redirects

Accrued Rewards - Low

Description:

In **ArbiFluxToken**, `mintToAddress` allows the minter recorded at lock time to mint the source position's accrued rewards to an arbitrary target, including the minter itself. The only gate is that the caller equals the stored minter address; the reward destination is otherwise unconstrained.

```
function mintToAddress(address sourceAddress, address targetAddress, uint256
targetBlock) ... public {
    // ...
    require(sourceAddressLock.minterAddress == _msgSender(), "You must be the
delegated minter of the sourceAddress");
    // ...
    _mint(targetAddress, mintAmount, "", "");
}
```

A locker who delegates minting to a third-party automation grants that delegate the ability to direct all mintable rewards. If the delegate is malicious or later compromised, it calls `mintToAddress` with itself as the target and captures every future accrual until the locker unlocks and re-locks with a different minter. The locker controls only who the minter is, set once at lock and changeable only by unlock followed by re-lock.

Relatedly, `lock` accepts any `minterAddress` including `address(0)` with no validation. Because `mintToAddress` requires `minterAddress == _msgSender()` and no call has `_msgSender() == address(0)`, a position locked with `minterAddress = 0` can never mint and its accrued rewards become permanently unmintable (principal stays recoverable via `unlock`).

Principal remains safe because the minter cannot unlock the victim, but all accrued rewards up to reconfiguration can be stolen.

Assets:

- `arbiFlux.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Constrain the target to the source address, or to a locker-approved allowlist, so the minter controls only timing and not destination. Alternatively, document prominently that delegating a minter grants full control of the reward destination and that only trusted automation should be delegated.

Additionally, add `require(minterAddress != address(0))` in `lock`.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** to ensure maximum flexibility for validators and integration smart contracts.

1. Flexible Reward Routing

In the Datamine ecosystem, the delegated minter has the absolute freedom to direct rewards to any target address:

- **Custom Routing:** This allows users to mint rewards directly to cold storage, smart contracts, multisigs, or even burn them immediately on claim.
- **Smart Contract Integration:** Custom batch minters and auto-compounding vaults rely on this unconstrained routing to execute compound locking or swap rewards in a single step. Restricting the mint target to the locking EOA would break these helper integrations.

2. Delegated Trust Model

When a user calls `lock()`, they explicitly designate the `minterAddress`. This represents a formal delegation of trust:

- The user is expected to delegate only to a trusted address, or to a custom contract they control.
- The principal locked tokens remain completely secure: the delegated minter has no permission to unlock or withdraw the user's locked tokens.
- If a delegation needs to be modified or revoked, the owner simply calls `unlock()` and re-locks their tokens with a new delegate.

3. Purposeful address(0) Support

Allowing `minterAddress = address(0)` is a valid and intentional configuration. If a user wishes to lock tokens purely to participate in the ecosystem

statistics (e.g. supporting locking metrics) but wants to opt-out of reward inflation entirely, they can delegate to `address(0)` to permanently disable minting rewards for that lock duration.

4. Contract Immutability

The deployed smart contracts are fully decentralized, ownerless, and immutable on-chain, and have operated successfully under this delegated minter architecture since their inception.

[F-2026-17955](#) - Misleading burnToAddress Function Transfers Tokens to Vault Instead of Burning - Low

Description:

In **LockquidityToken**, the `burnToAddress` function does not burn tokens despite its name and documentation. The function transfers LOCK tokens to a vault address using `_send` instead of destroying them with `_burn`. The inline comment incorrectly states "this will destroy LOCK tokens."

```
function burnToAddress(address targetAddress, uint256 amount) public {
    // ...
    targetAddressLock.burnedAmount = targetAddressLock.burnedAmount.add(amount);
    globalBurnedAmount = globalBurnedAmount.add(amount);

    emit BurnedToAddress(_msgSender(), targetAddress, amount);

    // Comment says "destroy" but code does transfer
    _send(_msgSender(), address(_vaultAddress), amount, "", "", false);
}
```

In contrast, **ArbiFluxToken** correctly implements token destruction:

```
_burn(_msgSender(), amount, "", "");
```

The `burnedAmount` tracking operates correctly for reward multiplier calculations. However, `totalSupply` never decreases since tokens are transferred rather than destroyed. This creates a discrepancy between the documented behavior and actual implementation, affects token economics, and may mislead integrators relying on the function name and comments.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Replace the `_send` call with `_burn` to match the intended behavior and the reference implementation:

```
_burn(_msgSender(), amount, "", "");
```

If the vault accumulation is intentional by design, rename the function to `transferToVault` or similar, update the documentation, and emit a distinct event to accurately reflect the operation.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** fundamental to the unique liquidity-backing mechanics of the Lockquidity (LOCK) token.

1. Permanent Liquidity Routing ("Proof of Liquidity")

Unlike the L1 FLUX token (which is purely deflationary and burns tokens permanently from the supply), LOCK was designed to act as a **stability and permanent liquidity asset**:

- **The Zap Mechanism:** When a user "burns" LOCK via `burnToAddress()`, the contract routes these tokens to the `LockquidityVault`.
- **Providing Depth:** The vault's `sweep()` function swaps half of the received LOCK for ETH and adds both back into the Uniswap V2 pool. This creates permanent protocol-owned liquidity, backing the LOCK token with real ETH value.
- **Why `**_send**` is Used:** Using `_burn` would destroy the tokens, making it impossible for the vault to swap them or pair them with ETH in the Uniswap pool. Thus, routing the tokens to the vault via `_send` is required to fund the permanent liquidity pool.

2. Code Comment Discrepancy

The inline comment stating *"this will destroy LOCK tokens"* is a leftover copy-paste artifact from the L1 `FluxToken.sol` contract foundation. While the comment itself is incorrect and should have been updated during the rewrite, the logic itself—transferring to the vault instead of burning—is the correct, intended behavioral architecture.

3. Contract Immutability

The `LockquidityToken` contract is deployed as an immutable protocol on Arbitrum (L2) with **no admin controls, owner keys, or upgrade path**.

Consequently, this permanent liquidity routing is an unchangeable rule of the L2 tokenomics.

F-2026-17962 - LockquidityVault init Function Lacks Access Control

- Low

Description:

The `init` function in **LockquidityVault** can be called by any address with any token parameter. While the **LockquidityFactory** deploys and initializes the vault atomically in its constructor, if the vault contract is deployed independently or the factory deployment fails partway, the `init` function remains vulnerable to front-running.

```
function init(address token) public {  
    require(!isInitialized);  
    _token = IERC777(token);  
    isInitialized = true;  
}
```

An attacker monitoring the mempool could front-run the legitimate initialization with a malicious token address. Once initialized, the token cannot be changed. The vault would then operate with an attacker-controlled token, potentially redirecting all `sweep` operations to benefit the attacker.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Set the token address in the constructor rather than requiring a separate initialization step, or add access control.

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted and mitigated design choice** of the deployment setup.

1. Atomic Initialization in Factory Constructor

The `LockquidityVault` is designed to be deployed exclusively through the `LockquidityFactory` contract. In the factory's constructor, the vault is created and immediately initialized in the exact same execution frame:

```
    vault = new LockquidityVault();  
    token = new LockquidityToken(...);  
    vault.init(address(token));
```

Because these calls occur atomically within the constructor transaction, there is **no temporal gap** between the vault's deployment and its initialization. An attacker cannot inject a transaction to front-run `init()` because the contract does not yet exist on-chain before this block executes.

2. Already Deployed and Initialized

The vault and factory are already deployed on Arbitrum L2, and the initialization has been successfully completed. Because the initialization is a one-time operation (`require(!isInitialized)`), it is permanently locked and cannot be altered or hijacked.

[F-2026-17971](#) - Integer Division Truncation in getMintAmount Results in Zero Rewards for Small Lock Amounts - Low

Description:

In **LockquidityToken** and **ArbiFluxToken**, the `getMintAmount` function calculates minting rewards using several scaling divisions. The order of operations can truncate fractional rewards to zero for small lock amounts or short minting intervals.

```
uint256 blocksMintedByAmount = amount.mul(blocksMinted);
uint256 fluxAfterMultiplier = blocksMintedByAmount.mul(burnMultiplier).div(_percentMultiplier).mul(timeMultiplier).div(_percentMultiplier);
uint256 actualFluxMinted = fluxAfterMultiplier.div(_mintPerBlockDivisor);
```

With `_percentMultiplier = 10000` and `_mintPerBlockDivisor = 108`, the final division requires `fluxAfterMultiplier >= 108` for a non-zero mint amount. At minimum intended multipliers, this requires `amount * blocks >= 108` wei. Earlier divisions by `_percentMultiplier` discard precision before the final mint divisor is applied, so small valid locks can accumulate less than one whole reward unit for a given mint window and receive no LOCK rewards. The user must wait longer or lock more tokens to cross the integer threshold, while gas is still spent on failed zero-mint attempts.

Additionally, early divisions discard precision that could be preserved by reordering operations. The current formula performs two divisions before the final scaling, each truncating intermediate results. Multiplying all numerators first and dividing once at the end would retain maximum precision.

Assets:

- `arbiFlux.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- `lockquidity.sol` [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Medium

Recommendations

Remediation:

Reorder arithmetic operations to preserve precision by multiplying before dividing:

```
uint256 blocksMintedByAmount = amount.mul(blocksMinted);
uint256 numerator = blocksMintedByAmount.mul(burnMultiplier).mul(timeMultiplier);
uint256 denominator = _percentMultiplier.mul(_percentMultiplier).mul(_mintPerBlockDivisor);
uint256 actualFluxMinted = numerator.div(denominator);
```

Consider adding a minimum lock amount or minimum mint interval check to prevent dust locks from producing zero rewards.

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted architectural trade-off** in the protocol's mathematical design.

1. Overflow Prevention

The smart contracts were developed in Solidity v0.6.9. At the time of design, performing multiple large multiplications (`amount * blocksMinted * burnMultiplier * timeMultiplier`) in a single step carried a significant risk of integer overflow (exceeding `uint256` max) for validators holding substantial balances over multi-year timelines. By performing step-wise intermediate divisions, the contract ensures that numbers stay within safe bounds, completely eliminating overflow risk.

2. State Bloat & Dust Mitigation

Discarding fractional precision for extremely small "dust" positions (e.g., locking negligible amounts of wei or attempting to mint after only a few seconds) is a deliberate mechanism to prevent network spam and state bloat.

- **The Mathematical Threshold:** For a non-zero reward, the division requires `amount * blocks >= 108` (wei-equivalent units). Since 1 token (LOCK or ArbiFLUX) is 10^{18} wei, a standard lock of even **1 token** (10^{18} wei) clears this threshold on the very first block ($10^{18} \cdot 1 \gg 10^8$).
- **Real-World Metrics:** A lock must be less than (0.0000000001) tokens (10^8 wei) to experience truncation. In the actual production ecosystem:
 - **LOCK Current Supply:** 30,131.51096831 LOCK / \$71,326.96 USD

- **ArbiFLUX Available Liquidity:** 60,305.43 ArbiFLUX Normal user locks are typically in the hundreds or thousands of tokens, which sit 12 to 15 orders of magnitude above this dust limit. Therefore, this precision loss has zero impact on any legitimate validator.
- It discourages users from wasting gas on economically sub-optimal operations.

3. Contract Immutability

The deployed smart contracts are completely decentralized and immutable, with **no admin keys or upgrade path**. This precision behavior is a permanent rule of the ecosystem, and has successfully operated without issue for 5+ years on Ethereum L1 and over 1 year and 262 days on Arbitrum L2.

[F-2026-17943](#) - Inconsistent, Floating, and Outdated Pragma Versions Reduce Code Quality and Portability - Info

Description:

The codebase exhibits three pragma-related issues. First, embedded OpenZeppelin contracts use a floating pragma (`^0.6.0`) while custom contracts use fixed versions, creating inconsistency within the same file. Second, the fixed versions themselves differ across contracts: **ArbiFluxToken** and **LockquidityToken** specify `0.6.9`, while **Flux** specifies `0.6.11`. Third, all versions target the Solidity 0.6.x branch, which is significantly outdated; the current stable branch is 0.8.x with important security improvements including built-in overflow checks, custom errors, and improved optimizer behavior.

```
// fluxL2.sol - mixed pragmas
pragma solidity ^0.6.0; // OpenZeppelin contracts
pragma solidity 0.6.11; // Flux contract

// arbiFlux.sol - mixed pragmas
pragma solidity ^0.6.0; // OpenZeppelin contracts
pragma solidity 0.6.9; // ArbiFluxToken contract

// lockquidity.sol - mixed pragmas
pragma solidity ^0.6.0; // OpenZeppelin contracts
pragma solidity 0.6.9; // LockquidityToken contract
```

Floating pragmas can lead to contracts being compiled with different compiler versions in different environments, potentially introducing subtle behavioral differences. The outdated compiler version lacks modern safety features and may contain known bugs that have been fixed in later releases.

Assets:

- fluxL2.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- arbiFlux.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:

1/5

Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Use a consistent, fixed pragma version across all contracts. If upgrading to Solidity 0.8.x is feasible, migrate to benefit from built-in overflow protection (eliminating the need for SafeMath), custom errors for gas-efficient reverts, and other security improvements. If migration is not feasible, at minimum unify all contracts to the same 0.6.x fixed version.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted compiler configuration** with no security risk to the deployed protocol.

1. Locked Build Environments

While the pragma directives in the source files are mixed or floating, the actual compiler versions were strictly locked and defined in the Hardhat and compilation build configuration files during deployment.

- This ensured that the compiled bytecode was generated under deterministic compiler versions (e.g., 0.6.9/0.6.11) with identical optimization parameters, preventing any environment-specific compilation behavior changes.

2. Historical Context & Parity

The protocol was launched during the Solidity 0.6.x era. When deploying ArbiFLUX and LOCK on Arbitrum, retaining the 0.6.x branch was important to maintain logic, feature, and typing parity with the L1 codebase, avoiding compile-time signature or behavior discrepancies.

3. Contract Immutability

All contracts are deployed on Arbitrum L2 as fully decentralized, ownerless, and immutable contracts with **no upgrade path**. As a result, this compiler configuration is permanent and cannot be modified.

[F-2026-17947](#) - Misleading NatSpec Comment on `_percentBurnMultiplier` Constant - Info

Description:

In **LockquidityToken**, the `_percentBurnMultiplier` constant is set to `1` and is used in `getAddressBurnMultiplier` as the additive floor of the burn multiplier and as the value returned when `globalRatio == 0` or `myRatio == 0`. The value is intentional: unlike `_percentMultiplier` (`10000`, the four-decimal `1.0x` scale applied to the ratio term), `_percentBurnMultiplier` is a raw additive base, and the implementation is correct for the intended burn-multiplier semantics.

```
/**
 * @dev To get 4 decimals on our multipliers we'll multiply all ratios & divide ratios by this number.
 * @dev This is done because we're using integers without any decimals.
 */
uint256 private constant _percentBurnMultiplier = 1;
```

The NatSpec above `_percentBurnMultiplier` is copied verbatim from `_percentMultiplier` and states the constant is used "to get 4 decimals on our multipliers [by multiplying and dividing] ratios by this number." That description does not apply to a raw value of `1`; taken at face value it implies the constant should be `10000` (as in **ArbiFluxToken**), which can mislead readers, integrators, and reviewers into treating a correct value as a defect.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Replace the copied NatSpec on `_percentBurnMultiplier` with an accurate description of its role as the raw additive floor and zero-ratio base of the burn multiplier, explicitly distinguishing it from the four-decimal `_percentMultiplier`.

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** crucial to the unique tokenomics and liquidity-incentive model of Lockquidity (LOCK).

1. Forcing Contribution to the Permanent Liquidity Pool

Unlike the ArbiFLUX token, the core mission of Lockquidity is to act as a stability and permanent liquidity token:

- **The Core Mechanism:** When LOCK is burned, the contract automatically sells half of the burned LOCK for Ethereum and adds both back into a permanent, decentralized Uniswap liquidity pool.
- **Why the Floor is $\backslash(0.0001\backslash\text{text}\backslash)$ (`**_percentBurnMultiplier = 1**`):** If the floor multiplier for non-burners was set to $\backslash(1.0\backslash\text{text}\backslash)$ (10000), users could lock ArbiFLUX and earn standard LOCK rewards indefinitely without ever contributing to the liquidity pool.
- **The Burning Incentive:** By setting `_percentBurnMultiplier = 1` (a floor of $\backslash(0.0001\backslash\text{text}\backslash)$), earning meaningful rewards is restricted *exclusively* to validators who actively burn LOCK. This structure transforms burning from an optional boost into a core requirement. It ensures that 100% of validators actively fund the permanent liquidity pool, driving price stability and building deep, decentralized market reserves.

2. Immutability on Arbitrum (L2)

- The LockquidityToken contract is deployed on Arbitrum as a fully decentralized, immutable protocol (**no owner keys, no admin roles, and no upgradeability**).
- Even if this model diverges from the L1 ArbiFLUX tokenomics, it is an immutable law of Lockquidity on L2. The ecosystem has fully adapted to this design, successfully driving inflation down from 700% to 122% while accumulating over \$135,000 in permanent pool liquidity.
- **Code Comment Discrepancy:** The inline comment in `LockquidityToken.sol` describing `_percentBurnMultiplier = 1` was copy-pasted directly from the L1 FLUX token implementation codebase (where `_percentMultiplier = 10000` is used). While the comment itself is a leftover artifact from that base code, the chosen value of `1` remains the correct, intended economic multiplier floor for Lockquidity tokenomics.

Evidences

PoC: Incorrect `_percentBurnMultiplier` Constant in `LockquidityToken`

Reproduce:

PoC Steps:

1. Deploy mock contracts replicating the math logic from `LockquidityToken` (buggy) and `ArbiFluxToken` (correct)
2. Call `getAddressBurnMultiplier(0, 0)` on both contracts to verify default return values
3. Calculate minting rewards using both multipliers for identical lock amounts
4. Assert that buggy version returns exactly 10000x less rewards

```
function test_MintRewardsAre10000xLowerForNonBurners() public {
    uint256 lockedAmount = 1000 * 1e18;
    uint256 blocksMinted = 1000;
    uint256 timeMultiplier = 10000;

    uint256 buggyBurnMultiplier = buggy.getAddressBurnMultiplier(0, 0);
    uint256 correctBurnMultiplier = correct.getAddressBurnMultiplier(0, 0);

    uint256 buggyMint = buggy.getMintAmount(
        lockedAmount, blocksMinted, buggyBurnMultiplier, timeMultiplier
    );
    uint256 correctMint = correct.getMintAmount(
        lockedAmount, blocksMinted, correctBurnMultiplier, timeMultiplier
    );

    uint256 lossRatio = correctMint / buggyMint;

    assertGt(correctMint, 0, "Correct mint should be positive");
    assertGt(buggyMint, 0, "Buggy mint is positive but 10000x smaller");
    assertEq(lossRatio, 10000, "Users receive exactly 10000x less rewards");
}

function test_EarlyProtocolUsersGet10000xLessRewards() public {
    uint256 lockedAmount = 100 * 1e18;
    uint256 blocksMinted = 5760;
    uint256 timeMultiplier = 10000;

    uint256 buggyMint = buggy.getMintAmount(
        lockedAmount,
        blocksMinted,
        1,
```

```

        timeMultiplier
    );

    uint256 correctMint = correct.getMintAmount(
        lockedAmount,
        blocksMinted,
        10000,
        timeMultiplier
    );

    assertGt(correctMint, 0, "Correct: users get positive rewards");
    assertEq(correctMint / buggyMint, 10000, "Loss is exactly 10000x");
}

function test_NonBurnersLose99990f10000Rewards() public {
    uint256 lockedAmount = 10000 * 1e18;
    uint256 blocksMinted = 161280;
    uint256 maxTimeMultiplier = 30000;

    uint256 buggyMint = buggy.getMintAmount(
        lockedAmount,
        blocksMinted,
        1,
        maxTimeMultiplier
    );

    uint256 correctMint = correct.getMintAmount(
        lockedAmount,
        blocksMinted,
        10000,
        maxTimeMultiplier
    );

    uint256 lossRatio = (correctMint - buggyMint) * 10000 / correctMint;

    assertGt(correctMint, buggyMint, "Correct mint > buggy mint");
    assertGe(lossRatio, 9999, "Loss is at least 99.99%");
}

```

Results:

```

[PASS] test_BurnMultiplierReturns1InsteadOf10000() (gas: 17495)
[PASS] test_MintRewardsAre10000xLowerForNonBurners() (gas: 36121)

```

```
[PASS] test_EarlyProtocolUsersGet10000xLessRewards() (gas: 30530)
[PASS] test_NonBurnersLose99990f10000Rewards() (gas: 30978)
```

Files:

PoC_LockquidityBurnMultiplier.t.sol

[F-2026-17957](#) - Incorrect Token Reference in Code Comment

Reduces Code Clarity - Info

Description: In **LockquidityToken**, the `lock` function contains a comment that incorrectly references "DAM token" instead of "ArbiFLUX token."

```
// Send [amount] of DAM token from the address that is calling this function
to LOCK smart contract.
IERC777(_token).operatorSend(_msgSender(), address(this), amount, "", "");
```

The **LockquidityToken** contract accepts ArbiFLUX tokens (not DAM tokens) for lock-in operations. This comment appears to be a copy-paste artifact from the original FLUX contract documentation. While the code functions correctly, the misleading comment may confuse developers during maintenance or audit, potentially leading to incorrect assumptions about token flows.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status: Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Update the comment to accurately reflect the token type:

```
// Send [amount] of ArbiFLUX token from the address that is calling this func
tion to LOCK smart contract.
IERC777(_token).operatorSend(_msgSender(), address(this), amount, "", "");
```

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted documentation artifact** with no functional impact on the protocol's execution.

1. Zero Runtime Impact

Comments in Solidity are completely discarded during compilation and are not included in the compiled EVM bytecode.

- The typo ("DAM token" instead of "ArbiFLUX token") has **no influence** on contract safety, gas consumption, or execution correctness.
- The EVM executes the `operatorSend()` function targeting the correct L2 token address loaded in the `_token` variable during deployment (which correctly references ArbiFLUX).

2. Copy-Paste Foundation

`LockquidityToken` was developed using the L1 `FluxToken` implementation codebase as a foundation. Because `FluxToken` locks DAM on L1, this inline comment was carried over as a minor copy-paste artifact.

3. Contract Immutability

The token contract is deployed on Arbitrum L2 as a fully decentralized, ownerless, and immutable protocol with **no upgrade path**. As a result, this comment is permanent and cannot be modified.

[F-2026-17959](#) - Vestigial ALREADY_INIT Guard Is Always True in Constructor - Info

Description:

In **Flux**, the constructor assigns the gateway address and includes a guard requiring the gateway storage variable to equal the zero address before assignment. Because constructor storage is always zero-initialized and the constructor runs once, this guard is unconditionally true and protects nothing.

```
constructor(  
    address _l2Gateway,  
    address _l1Counterpart  
) public ERC777("Flux", "FLUX", new address[](0)) {  
    require(_l2Gateway != address(0), "INVALID_GATEWAY");  
    require(l2Gateway == address(0), "ALREADY_INIT");  
    // ...  
}
```

The guard is an initializer-style re-initialization check placed in a constructor context. Under direct deployment it is harmless dead code with no effect on on-chain behavior.

Assets:

- fluxL2.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Remove the redundant requirement for a pure-constructor deployment.

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted code artifact** with no functional impact on on-chain behavior.

1. Harmless Dead Code

The check `require(l2Gateway == address(0), "ALREADY_INIT")` is an initializer-style re-initialization guard placed in the constructor. While it is redundant under direct constructor execution because constructor storage is always zero-initialized, it is completely harmless and does not affect the safety or logic of token operations.

2. Contract Immutability

The `Flux` token contract is already deployed on Arbitrum L2 as a fully decentralized, ownerless, and immutable contract with **no upgrade path**. As a result, this redundant check is permanent.

[F-2026-17961](#) - Missing SafeMath in LockquidityVault Arithmetic Operations - Info

Description:

The **LockquidityVault** contract performs arithmetic operations in `getSwapAmount` and `sqrt` functions without using the **SafeMath** library.

```
function getSwapAmount(uint256 r, uint256 a) public pure returns (uint256) {
    return (sqrt(r * (r * 3988009 + a * 3988000)) - r * 1997) / 1994;
}

function sqrt(uint256 y) private pure returns (uint256 z) {
    if (y > 3) {
        z = y;
        uint256 x = y / 2 + 1;
        while (x < z) {
            z = x;
            x = (y / x + x) / 2;
        }
    }
    // ...
}
```

While the rest of the codebase consistently uses SafeMath for arithmetic operations, these functions use native Solidity arithmetic. In Solidity 0.6.x, native arithmetic does not include overflow checks. For extremely large reserve values, expressions like `r * r * 3988009` could theoretically overflow, though this scenario is unlikely in practice with typical Uniswap reserve sizes.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple

Recommendations

Remediation: Use **SafeMath** for consistency and safety.

Resolution: The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted trade-off** with no real-world risk.

1. Inherent Overflow Safety

For an overflow to occur in $r * (r * 3988009 + a * 3988000)$, the calculated value must exceed the maximum value of a 256-bit integer ($(2^{256} - 1) \approx 1.15 \times 10^{77}$).

- In practice, the total circulating supply of LOCK is **30,131.51 LOCK** ($\approx 3 \times 10^{22}$ wei). Even if the entire supply was locked in the liquidity pool, the calculations remain more than 25 orders of magnitude below the overflow limit.
- Therefore, under all normal/expected cases, overflow is mathematically impossible.

2. Revert Behavior

In the highly theoretical event that reserve metrics did somehow scale astronomically to trigger an overflow or underflow (such as inside the `sqrt()` subtraction), the transaction would automatically revert (e.g. from array operations, out of gas, or subtraction underflow checks in the Uniswap router), protecting the contract state.

3. Contract Immutability

The **LockquidityVault** is already deployed on Arbitrum L2 as a fully decentralized, ownerless, and immutable contract with **no upgrade path**. The math functions are locked and have functioned without error.

F-2026-17963 - Misleading Variable Name damBalance in View Functions - Info

Description: Both **ArbiFluxToken** and **LockquidityToken** contain a view function `getAddressTokenDetails` that uses the variable name `damBalance` to refer to different token balances.

```
// In ArbiFluxToken - refers to FLUX tokens
function getAddressTokenDetails(address targetAddress) public view returns(..
.) {
    uint256 damBalance = IERC777(_token).balanceOf(targetAddress);
    // ...
}

// In LockquidityToken - refers to ArbiFLUX tokens
function getAddressTokenDetails(address targetAddress) public view returns(..
.) {
    uint256 damBalance = IERC777(_token).balanceOf(targetAddress);
    // ...
}
```

The variable name "damBalance" appears to be a historical artifact from the original Datamine (DAM) token contracts. In **ArbiFluxToken**, the `_token` reference points to FLUX, while in **LockquidityToken**, it points to ArbiFLUX. Using "damBalance" in both contexts creates confusion about which token balance is being retrieved.

Assets:

- arbiFlux.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Recommendations

Remediation:

Rename the variables to accurately reflect the token types:

```
// In ArbiFluxToken
uint256 fluxBalance = IERC777(_token).balanceOf(targetAddress);

// In LockquidityToken
uint256 arbiFluxBalance = IERC777(_token).balanceOf(targetAddress);
```

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted naming artifact** with zero impact on the security or execution of the smart contracts.

1. Compiled to Stack Positions

In EVM execution, local variable names (`damBalance`) are completely stripped and compiled away into EVM stack operations and stack offsets.

- The name of the variable has **no influence** on gas consumption, bytecode size, execution correctness, or security.
- The compiler executes the underlying call (`balanceOf()`) correctly using the stored contract `_token` address, which correctly references FLUX (on L1) or ArbiFLUX (on L2) as defined in each contract's constructor.

2. Copy-Paste Foundation

The codebase for both `ArbiFluxToken` and `LockquidityToken` was developed using the L1 `FluxToken` contract structure as a template. The internal variable name `damBalance` was carried forward from that original codebase as a minor copy-paste artifact.

3. Contract Immutability

Both tokens are deployed as fully decentralized, ownerless, and immutable contracts on-chain with **no upgrade path**. The internal names of local variables cannot be altered, nor do they pose any functional or security risks to the ecosystem.

F-2026-17968 - State Variables in Flux Contract Should Be Declared Immutable - Info

Description:

In **Flux**, the state variables `l2Gateway` and `l1Address` are set once in the constructor and never modified, yet they lack the `immutable` keyword:

```
contract Flux is ERC777, IArbToken {  
    address public l2Gateway;  
    address public override l1Address;  
}
```

Regular state variables require SLOAD operations (2100 gas after EIP-2929) on every read. The `onlyGateway` modifier reads `l2Gateway` on each `bridgeMint` and `bridgeBurn` call:

```
modifier onlyGateway() {  
    require(msg.sender == l2Gateway, "ONLY_GATEWAY");  
    _;  
}
```

Declaring these variables as `immutable` embeds values directly in bytecode, reducing read cost to approximately 3 gas per access. For high-frequency bridge operations, this accumulates meaningful gas savings.

Assets:

- fluxL2.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Add the `immutable` keyword to both variables:

```
address public immutable l2Gateway;  
address public immutable override l1Address;
```

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted gas trade-off** with no security implications for the protocol.

1. Effectively Read-Only

Although `l2Gateway` and `l1Address` are not marked with the `immutable` keyword in Solidity, they are written once in the constructor and have no setter functions. Thus, they are effectively read-only and structurally secure against any post-deployment modification.

2. Parity and Timing of Deployment

During the initial design and testing phase of the L2 Arbitrum gateway integration in 2021, using standard state variables was the standard, tested layout pattern for custom gateway implementations.

3. Negligible User Gas Impact

The `onlyGateway` modifier and these variables are only accessed during `bridgeMint()` and `bridgeBurn()` calls (i.e., when assets are deposited/withdrawn across the L1-L2 Arbitrum bridge).

- These are not high-frequency operations for regular users.
- For all core user actions—such as locking DAM, burning tokens, or minting FLUX rewards—these bridge variables are never read, meaning regular users experience zero gas overhead from this setup.

4. Contract Immutability

The L2 `Flux` token contract is deployed on Arbitrum as an ownerless, immutable contract with **no upgrade path**. As a result, this state variable configuration is permanent.

F-2026-17970 - Redundant Array Allocation in `_swap` Function - Info

Description:

The `_swap` function in **LockquidityVault** allocates a memory array twice in consecutive lines. The first allocation is immediately discarded when the second allocation overwrites the variable.

```
function _swap(address _from, address _to, uint256 _amount) internal {  
    IERC20(_from).approve(ROUTER, _amount);  
  
    address[] memory path = new address[](2); // First allocation (wasted)  
    path = new address[](2); // Second allocation (overwrites)  
  
    path[0] = _from;  
    path[1] = _to;  
    // ...  
}
```

The first `new address[](2)` allocates memory that is never used, wasting approximately 200 gas per `_swap` call.

Assets:

- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Remove the redundant allocation:

```
function _swap(address _from, address _to, uint256 _amount) internal {  
    IERC20(_from).approve(ROUTER, _amount);  
}
```

```
address[] memory path = new address[](2);
path[0] = _from;
path[1] = _to;
// ...
}
```

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **accepted code artifact** with no functional impact on the protocol's execution.

1. Negligible Cost and Impact

The redundant array allocation in the `_swap` function of the `LockquidityVault` is a minor copy-paste typo in the source code layout.

- **Gas Impact:** Wasting ~200 gas is mathematically and economically negligible, especially on Arbitrum (L2) where transaction fees are extremely low (amounting to a fraction of a cent).
- **Zero Security Risk:** This duplicate allocation has no impact on safety, swap execution, routing correctness, or liquidity generation.

2. Contract Immutability

The `LockquidityVault` is already deployed on Arbitrum L2 as a fully decentralized, ownerless, and immutable contract with **no upgrade path**. As a result, this minor gas overhead is permanent and cannot be modified.

F-2026-17972 - Incomplete Same-Block Guard in unlock Allows Immediate Relock - Info

Description: The `preventSameBlock` modifier enforces that `lock`, `unlock`, and `mintToAddress` cannot be called in the same block by checking `blockNumber` and `lastMintBlockNumber`. However, the `unlock` function does not update either timestamp field after execution.

```
function unlock() preventReursion preventSameBlock(_msgSender()) requireLocked(_msgSender(), true) public {
    AddressLock storage senderAddressLock = addressLocks[_msgSender()];
    uint256 amount = senderAddressLock.amount;
    senderAddressLock.amount = 0;
    // blockNumber and lastMintBlockNumber are NOT updated
    globalLockedAmount = globalLockedAmount.sub(amount);
    globalBurnedAmount = globalBurnedAmount.sub(senderAddressLock.burnedAmount);
    IERC777(_token).send(_msgSender(), amount, "");
}
```

If a user locked at block N and unlocks at block M, `blockNumber` remains N. A second transaction in block M can call `lock` because the check `blockNumber != block.number` passes (N != M).

Assets:

- arbiFlux.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]
- lockquidity.sol [<https://github.com/Datamine-Crypto/white-paper/tree/master>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Update `blockNumber` to the current block in `unlock` to maintain the same-block protection invariant:

```
function unlock() preventReursion preventSameBlock(_msgSender()) requireLock
ed(_msgSender(), true) public {
    AddressLock storage senderAddressLock = addressLocks[_msgSender()];
    uint256 amount = senderAddressLock.amount;
    senderAddressLock.amount = 0;
    senderAddressLock.blockNumber = block.number; // Add this line
    // ...
}
```

Resolution:

The team acknowledged and accepted the risk with the following comment:

This behavior is an **intentional design choice** designed to support user convenience (unlocking and immediately re-locking) while maintaining absolute system safety.

1. Enabling Single-Block Re-Locking and Test Coverage

If a validator wants to update their delegated minter address or modify their locked amount, they must call `unlock()` and then `lock()`.

- **The Single-Block UX:** Allowing a user to `unlock()` and immediately `lock()` again in the same block M is a beneficial design feature. It lets users reconfigure their positions without waiting for the next block to execute, saving gas and transaction overhead.
- **Why This is Safe:** Since the lock state is updated in the subsequent `lock()` call, the new `blockNumber` is set to block M. As a result, the user is still correctly prevented from calling `mintToAddress()` in block M (since the new `blockNumber == M` fails the `preventSameBlock` check).
- **Test Coverage:** We explicitly test the security and state persistence of unlock/re-lock sequences in our test suites:
 - `should not be possible to mint tokens for a past lock period after re-locking` (in `FluxTokenAttackScenariosTests.ts` and `LockTokenAttackScenariosTests.ts`) ensures that re-locking correctly updates block tracking and prevents claiming historical rewards.
 - `Should preserve burned amount across unlock/re-lock cycles (no free multiplier reset)` (in `FluxTokenAttackTests.ts` and `LockTokenAttackTests.ts`) validates correct state restoration across these transition cycles.

2. Multi-Unlock Prevention

Allowing the block tracking to remain unmodified on `unlock()` does not introduce any double-spend or double-unlock vulnerabilities:

- `unlock()` is protected by the `requireLocked(_msgSender(), true)` modifier, which asserts that `amount > 0`.
- Since `unlock()` immediately sets `amount = 0`, any secondary attempt to call `unlock()` in the same block will revert due to the zero-balance requirement, ensuring double-unlocking is structurally impossible.

3. Contract Immutability

The contracts are deployed as decentralized, immutable protocols on-chain, and this transition logic has operated securely and transparently in production.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Datamine-Crypto/white-paper
Commit	28c902ff56b1126228ec8dcbb5e244b9b6729842
Deployed Contracts	FLUX(L2) Deployed address: 0xF80D589b3Dbe130c270a69F1a69D050f268786Df
	ArbiFLUX Deployed address: 0x64081252c497FCfeC247a664e9D10Ca8eD71b276
	Lockquidity Deployed address: 0x454F676D44DF315EEf9B5425178d5a8B524CEa03
Whitepaper	https://github.com/Datamine-Crypto/white-paper/blob/master/README.md
Requirements	https://github.com/Datamine-Crypto/white-paper/tree/master/docs
Technical Requirements	https://github.com/Datamine-Crypto/white-paper/tree/master/docs

Asset	Type
arbiFlux.sol [https://github.com/Datamine-Crypto/white-paper/tree/master]	Smart Contract
fluxL2.sol [https://github.com/Datamine-Crypto/white-paper/tree/master]	Smart Contract
lockquidity.sol [https://github.com/Datamine-Crypto/white-paper/tree/master]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.